Scout Shopping Assistant

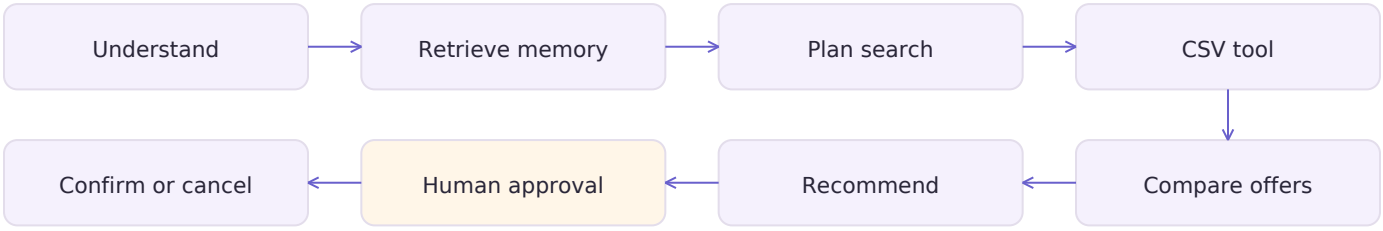
ITS 58100 | Project 1 | Team 1: Blas Moreno Laguna, Yasaa Usman Ahmad Khan, Mohammad Noman Hussain

Code repository: github.com/slab10000/scout-shopping-assistant

Scout helps a user choose wireless headphones by turning a natural-language request into a constrained search, a reasoned comparison, and a recommendation that waits for human approval. One Gemini-powered LangGraph workflow uses a CSV information source and approved preferences across separate interactions. The local web app has Shopping, Catalog, Memory, and Architecture tabs.

Example: “Find wireless headphones under $150 for studying; prioritize noise cancellation.” The sample catalog contains 110 fictional offers across 36 models and three simulated retailers. Output includes a shortlist, known item-and-shipping totals before tax, exclusions, uncertainty, and a proposed simulated purchase.

Workflow and execution control



Separate memory path: Form preference > Human approval > Save or finish without a write

Conditional paths: clarify missing budget; stop unsupported requests or unmatched offers.

Full graph: Architecture tab, architecture.svg, and the generated architecture.mmd file.

The implementation follows the labs: TypedDict state, small node functions, partial updates, conditional edges, and an audit log. Gemini extracts requirements, writes the search strategy, compares the shortlist, recommends one offer, and proposes memories. Python binds the validated filters to the plan and enforces schemas, decimal arithmetic, and allowed offer IDs.

Comparison and recommendation are two separate Gemini calls. compare_offers sends the request, applied preferences, all CSV search results, and the validated shortlist; Gemini returns two to four bullets per offer plus overall takeaways. recommend receives that comparison and writes one paragraph on why the chosen offer beats the alternatives. Python rejects invented, missing, or duplicate offer IDs.

The graph pauses with interrupt() before a simulated purchase or memory write. A strict boolean passed through Command(resume=...) selects approval or rejection in the same thread, and the side effect runs in a later node that checks approval again. No real order or payment exists.

Design decisions

Current constraints override remembered defaults. Unknown shipping cannot win a delivered-cost comparison, and only identical model, variant, and condition offers are interchangeable. No-match requests stop without relaxing the budget, and Gemini errors never silently switch to the sample engine.

Thread state and reusable memory

InMemorySaver checkpoints each thread_id: request, constraints, plan, offers, approval, status, and events. InMemoryStore holds approved records under the user’s shopping namespace. A new thread retrieves relevant active records without inheriting the old approval or execution state. Both backends clear on process restart.

Memory formation separates STORE candidates, TEMPORARY context, and HUMAN_REVIEW. Only supported, canonical fit, condition, feature, and ranking preferences can be saved after approval. Ranking uses relevance, importance, and deterministic timestamp-based recency. A correction supersedes the old

active record while preserving its source, approval time, and replacement link.

Recorded scenarios

Scenario	Expected behavior	Observed sample result
Normal request	Pause, then confirm simulation	SIMULATED_CONFIRMED
Later thread with two records	Apply over-ear and new-only preferences	2 records; O072 selected
Rejected purchase	No confirmation node	CANCELLED
Approved correction	Replace new-only with any condition	any
Temporary exception	No reusable write	NOT_STORED
Rejected memory	Keep store unchanged	MEMORY_REJECTED
No suitable offer	Stop without action	NO_MATCHES

31 local regression checks pass with the labeled sample engine, covering graph pauses, strict consent, money totals, unknown shipping, memory correction, temporary exceptions, scoped access, the comparison handoff, and invented offer IDs. Demo decisions are scripted mock-human inputs through real interrupts.

Live Gemini runs (gemini-3.1-flash-lite) passed the normal, two-memory, retrieval, rejection, correction, and no-match scenarios on the original catalog. A later live run verified the comparison handoff with all 110 search results and two applied preferences (evidence/llm-comparison). Some attempts hit provider quota or high-demand errors.

Team 1 contributions

Member	Part	Main responsibilities
Blas Moreno Laguna	Agent workflow and Gemini	LangGraph state and routing; Gemini understanding, planning, comparison, and recommendation; purchase approval
Yasaa Usman Ahmad Khan	Reusable memory	Memory policy and approval; user namespaces; ranking and retrieval; corrections and history; memory tests
Mohammad Noman Hussain	Catalog, web app, documentation	CSV catalog, search tool, and offer totals; web frontend; architecture diagram, notebook, report, and evidence

All three members shared the problem definition, workflow design review, scenario testing, and the live demonstration.

Instructor discussion and next steps

We reviewed Scout with the professor. The discussion focused on three directions for Project 2: replacing the fixed CSV with live product data through real APIs, adding more tools, and supporting multiple users with their own threads. Our plan follows the Week 8 and Week 9 material on role-based agents and shared memory.

- **Live search APIs.** Tavily web search for product pages and reviews, plus Rainforest API (Amazon) and BlueCart API (Walmart) for structured prices, stock, and shipping. Each result keeps its source, retailer, and timestamp so stale prices can be refreshed. The CSV stays as an offline fallback and test fixture.
- **More tools.** A price-history tracker that stores price snapshots to spot real discounts, a zip-code tax and shipping estimator, a review summarizer limited to retrieved review text, a coupon checker, and a simulated price-drop alert. Purchases remain simulated and approval-gated.
- **Collaborating agents.** A Planner that reads private preferences, a Researcher that calls the live APIs but cannot read private memory, and a Reviewer that checks prices and sources but cannot approve a purchase, with read and write permissions defined per role.
- **Multi-user threads and persistence.** At least two simulated shoppers, each with a private memory namespace and several threads, plus a shared deal board that holds only validated, user-approved records. One user cannot read another user’s private data. SQLite-backed memory and checkpoints will survive a restart.

Limitations

Scout is a local prototype: simulated user IDs are not real authentication, memory clears on restart, the catalog is fictional, and prices exclude tax. Live APIs, multiple agents, and shared memory are Project 2 work.

Appendix: Architecture diagram

The native LangGraph rendering from app.get_graph(), the same image shown in the web app's Architecture tab. Solid edges are fixed steps; dotted edges are conditional routes. Gemini runs in understand, plan_search, compare_offers, recommend, and form_memory. search_catalog is the CSV tool. retrieve_memory and save_memory read and write the user's InMemoryStore namespace, while InMemorySaver checkpoints each thread. clarify, approve_purchase, and approve_memory pause for a human decision.

